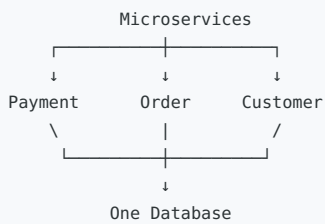


Database per Service Pattern

1. What Is the Database per Service Pattern?

In a microservices architecture, each service **owns its data and database schema**.

Instead of:



you aim for:

```
Payment Service → Payment DB
Order Service   → Order DB
Customer Service → Customer DB
```

The key principle is:

A service owns its data and other services access that data through the owning service's API/events, not by directly querying its database.

2. Why Do We Need It?

Imagine:

```
Payment Service
Order Service
Customer Service
    ↓
    Same DB
```

Now Order Service directly queries:

```
SELECT * FROM payment;
```

This creates tight coupling.

If Payment Service changes:

```
payment table
→ rename column
→ split table
→ change schema
```

Order Service can break.

The services are separate in deployment but still tightly coupled through the database.

That is often called a **distributed monolith** pattern.

3. With Database per Service

Instead:

```
Payment Service
  ↓
Payment DB

Order Service
  ↓
Order DB
```

Order Service cannot simply run:

```
SELECT * FROM payment_db.payment;
```

Instead:

```
Order Service
  ↓
Payment Service API
  ↓
Payment DB
```

or consume an event:

```
Payment Service
  ↓
PaymentCompleted
  ↓
Message Broker
  ↓
Order Service
```

This creates a stronger service boundary.

4. Is It Literally a Separate Database Server?

Not necessarily.

This is an important interview nuance.

“Database per service” means **logical ownership**, not necessarily:

“Every service must have a completely separate physical database server.”

Possible setups:

Separate database instances

```
Payment → DB Server 1
Order   → DB Server 2
```

Separate databases on same server

```
DB Server
├─ payment_db
├─ order_db
└─ customer_db
```

Separate schemas

```
DB Server
├─ payment_schema
└─ order_schema
```

The important part is:

Services should not bypass ownership boundaries and directly manipulate another service's data.

5. What Does “Own Its Data” Mean?

Suppose Payment Service owns:

```
Payment
Transaction
Refund
```

Then Payment Service is responsible for:

- schema
- writes
- business rules
- migrations
- data integrity

Another service needing payment information should ask Payment Service:

```
Order Service
  ↓
GET /payments/{id}
  ↓
Payment Service
```

rather than:

```
Order Service
↓
SELECT * FROM payment
```

6. The Biggest Advantage: Loose Coupling

With direct DB access:

```
Order Service
↓
Payment DB
```

Order is coupled to:

```
table names
columns
schema
indexes
database technology
```

With service ownership:

```
Order Service
↓
Payment API
↓
Payment DB
```

Order depends on the **API contract**, not internal database implementation.

That's the architectural advantage.

7. But You Lose Simple Joins

This is one of the biggest disadvantages.

Monolith:

```
SELECT *
FROM orders o
JOIN payments p
ON o.payment_id = p.id;
```

Easy.

With database-per-service:

```
Order DB    Payment DB
```

You can't normally do a simple cross-service SQL join.

Instead you may need:

```
Order Service
  ↓
Payment API
```

or:

```
PaymentCompleted event
  ↓
Order Service
  ↓
maintain local read model
```

This introduces more complexity.

8. How Do You Query Data Across Services?

There are several options.

Option 1: API call

```
Order Service
  ↓
Payment Service
  ↓
Payment DB
```

Good when you need current information.

Downside:

- network latency
- downstream dependency
- failure handling

Option 2: Event-driven replication

Payment Service publishes:

```
PaymentCompleted
```

Order Service consumes it:

```
PaymentCompleted
  ↓
Order Service
  ↓
local payment status
```

Now Order Service has the information it needs locally.

But the data becomes **eventually consistent**.

9. Eventual Consistency Connection

This is exactly where the previous topic comes back.

Suppose:

```
Payment DB
status = SUCCESS
```

Then event is published.

But Order Service hasn't consumed it yet:

```
Payment = SUCCESS
Order's local copy = PENDING
```

For a short period, they're inconsistent.

Eventually:

```
Payment = SUCCESS
Order = updated
```

This is **eventual consistency**.

So:

```
Database per Service
↓
No shared transaction
↓
Often eventual consistency
↓
May need Saga / events / Outbox
```

These concepts are closely connected.

10. Database per Service + Saga

Suppose:

```
Order Service → Order DB
Payment Service → Payment DB
Inventory Service → Inventory DB
```

A business transaction crosses all three.

You cannot simply:

```
@Transactional
```

and expect one transaction across all databases.

A Saga can coordinate:

```
Create Order
↓
```

```
Reserve Inventory
  ↓
Process Payment
```

If payment fails:

```
Payment FAILED
  ↓
Release Inventory
  ↓
Cancel Order
```

Each service has its own local transaction.

11. Database per Service + Outbox

Now another problem:

Payment Service updates its DB and needs to publish an event.

Naively:

```
Payment DB update
  +
Kafka publish
```

One can succeed while the other fails.

Outbox solves this:

```
Payment DB Transaction
├─ payment = SUCCESS
└─ outbox_event = PaymentCompleted
    ↓
  Publisher
    ↓
  Kafka
```

This is a common combination in microservice architecture.

12. Can Two Services Share the Same Database?

Technically yes.

Architecturally, if they freely access each other's tables, you've weakened the database-per-service boundary.

For example:

```
Payment Service ─┐
                  ↓
                Shared DB
                  ↑
Order Service ─┐
```

This can be acceptable as a transitional architecture or under specific constraints, but it removes much of the independence that microservices are supposed to provide.

A good interview answer is:

“Physical database separation isn’t the absolute requirement; strict data ownership and service boundaries are the important principles.”

13. Can One Service Read Another Service’s Database?

Normally:

No, not if you’re following the pattern strictly.

Bad:

```
Order Service
  ↓
Payment DB
```

Better:

```
Order Service
  ↓
Payment API
  ↓
Payment DB
```

or:

```
Payment event
  ↓
Order Service local model
```

14. What About Foreign Keys Across Databases?

Suppose:

```
Order DB
order.payment_id
```

and Payment Service owns:

```
Payment DB
payment.id
```

You generally can’t rely on a normal cross-database foreign key in the same way you would inside one database.

So referential integrity becomes partly a **service/application responsibility**.

This is one reason distributed data management becomes more difficult.

15. Different Database Technologies

Database-per-service also allows services to choose different technologies when justified.

Example:

```
Payment Service → PostgreSQL
Search Service  → Elasticsearch
Catalog Service → MongoDB
```

This is often called **polyglot persistence**.

But don't interpret it as:

"Every microservice should use a different database."

That adds operational complexity. Use different technologies only when they provide a real benefit.

16. Database Migration Independence

Suppose Payment Service needs:

```
payments.amount
    ↓
payments.amount
payments.currency
```

It can evolve its schema independently, provided its service API/event contracts remain compatible.

This is one of the major benefits of data ownership.

17. Distributed Transactions Become Harder

Consider:

```
Order DB
Payment DB
Inventory DB
```

You want:

```
All succeed
OR
all rollback
```

With separate databases, that is much harder.

This is why microservice architectures commonly move toward:

```
Local transactions
```

```
+
Saga
+
Events
+
Outbox
+
Idempotency
+
Reconciliation
```

instead of one giant transaction.

18. Reporting Is Harder

Suppose management wants:

```
Customer
+ Orders
+ Payments
+ Refunds
+ Products
```

In a monolith, one database query might join everything.

With database-per-service, you may need:

```
Data from multiple services
↓
Data pipeline / events
↓
Reporting database / warehouse
↓
Analytics
```

This is an important trade-off.

19. Caching and Read Models

Suppose a service frequently needs payment status.

Instead of calling Payment Service for every request:

```
Order Service
↓
Payment Service
```

you might maintain a local read model:

```
Payment Service
↓
PaymentCompleted event
↓
Order Service
↓
Local read model
```

Now reads are faster and the service is less dependent on the Payment Service being available.

Trade-off:

The local data can be temporarily stale.

20. Database Per Service vs Shared Database

This comparison is likely to be asked.

Shared DB	Database per Service
Simple joins	Cross-service queries are harder
Simple transactions	Distributed transactions are harder
Strong immediate consistency easier	Eventual consistency common
High schema coupling	Better ownership boundaries
Easier reporting	Reporting needs additional mechanisms
Simpler initially	More operational complexity
Services coupled to DB	Services interact through contracts

21. When Should You Use It?

Database-per-service makes strong sense when:

- services have clear domain boundaries
- independent deployment is important
- teams need autonomous ownership
- scaling requirements differ
- schema independence matters

A shared database can be reasonable when:

- system is small
- services aren't truly independent
- transactional simplicity is more important
- you're in a migration phase

Again:

Don't choose microservice patterns just because they're fashionable.

22. Interview Scenario

Interviewer:

“Order Service needs payment information. Can it directly query the Payment database?”

Strong answer:

“Not under a strict database-per-service architecture. The Payment Service owns its data, so Order Service should obtain the information through the Payment Service’s API or consume events published by Payment Service. Direct database access would couple Order Service to Payment’s internal schema and make independent evolution difficult.”

23. Interview Scenario

“Payment and Order are in separate databases. How do you maintain consistency?”

Answer:

“We can’t rely on a single local database transaction across both services. Depending on the business requirement, we can use a Saga to coordinate the workflow, events for asynchronous communication, and compensating actions for failures. The system may use eventual consistency rather than requiring both databases to become consistent at exactly the same instant.”

24. Interview Scenario

“How do you ensure an event isn’t lost after a DB update?”

Answer:

“A transactional Outbox can store the business change and the corresponding event in the same local database transaction. A separate publisher then sends the event to the broker. If publishing fails, the event remains in the outbox for retry.”

25. Important 3-Year-Level Distinction

Don’t say:

“Database per service means one database server per microservice.”

Better:

“It means each service owns its data and controls access to it. That can be implemented using separate database instances, databases or schemas depending on the required isolation.”

That’s a much more mature answer.

26. BPS Questions — Investigate Your Codebase

Now use the concept to inspect BPS:

1. What data does BPS own?
2. Does BPS have its own database/schema?
3. Which tables are exclusively owned by BPS?
4. Does BPS directly access another application’s database?
5. How does BPS obtain information owned by another service?
6. Does BHS have its own database?
7. Does BPS communicate with BHS through an API rather than directly accessing its DB?
8. How does BPS maintain consistency between its data and Majesco?
9. How are reconciliation scenarios handled when two systems disagree?
10. Are there any cross-service transactions that cannot be handled by one local `@Transactional` transaction?
11. Is there any event/outbox mechanism involved?
12. Which system is the source of truth for each important piece of payment data?

Interview-ready summary

“The database-per-service pattern means each microservice owns and controls its own data rather than allowing other services to directly access its database. This reduces database coupling and allows services to evolve their schemas independently. The trade-off is that cross-service joins and distributed transactions become harder, so APIs, events, local read models, Saga, eventual consistency and patterns such as Outbox are often used. The database doesn’t necessarily have to be a physically separate server; the important principle is ownership and isolation of data access.”

Remember the core rule

```
Service A
  ↓
Service B API / Event
  ↓
Service B
  ↓
Service B's DB
```

NOT

Service A

↓

Service B's DB

That single principle is the heart of **Database per Service**.